CHATBOT CIMA

FASE II / MANUAL DE DESPLIEGUE

Procedimiento remoto para publicar, verificar, actualizar y reactivar CHATBOT CIMA mediante GitHub, Docker y Render.

Aplicación	CHATBOT CIMA — clasificador NLP de consultas operativas
Despliegue	Render · Docker · FastAPI
Fecha de corte	31 de octubre de 2026

URL pública: https://chatbot-cima-fase2.onrender.com/

https://github.com/alfredosalazar-ai/deep-learning-experiments

1. Objetivo

Documentar el procedimiento utilizado para desplegar el prototipo CHATBOT CIMA sin instalaciones locales en el equipo del usuario. GitHub funciona como repositorio fuente; GitHub Actions ejecuta pruebas y construye el contenedor; Render construye y ejecuta el servicio web desde el Dockerfile publicado.

2. Arquitectura de despliegue

Capa	Responsabilidad
GitHub	Repositorio deep-learning-experiments, rama main, código fuente, modelo y workflow.
GitHub Actions	Runner ubuntu-24.04; descarga código, configura Python, instala dependencias, ejecuta pytest, construye y prueba el contenedor Docker.
Docker	Empaqueta FastAPI, frontend, dependencias y artefacto Regresion_Logistica.joblib.
Render Web Service	Construye desde GitHub y publica el servicio en una URL HTTPS.
FastAPI	Expone GET /, GET /health, POST /predict y documentación OpenAPI/Swagger.

3. Estructura relevante del repositorio

La carpeta FaseII/ contiene el contexto del servicio Docker. El workflow se encuentra en la raíz del repositorio, bajo .github/workflows/.

```
deep-learning-experiments/
├── .github/
│   └── workflows/
│       └── fase2-docker.yml
├── Actividad5/
├── Actividad6/
└── FaseII/
    ├── README.md
    ├── Dockerfile
    ├── requirements.txt
    ├── .dockerignore
    ├── backend/
    ├── frontend/
    ├── modelos/
    │   └── Regresion_Logistica.joblib
    ├── tests/
    └── docs/
```

4. Construcción y validación en GitHub Actions

Workflow: “Fase II - prueba y construcción Docker”. Archivo: .github/workflows/fase2-docker.yml. La configuración final utiliza runner ubuntu-24.04, actions/checkout@v7 y actions/setup-python@v7. La versión de Python configurada por el workflow debe conservarse alineada con las dependencias de pruebas y con el runtime del contenedor.

Paso del workflow	Qué verifica
Set up job / Descargar código fuente	Prepara el runner y obtiene el commit de la rama main.
Configurar Python	Prepara el intérprete requerido por las pruebas.
Instalar dependencias de prueba	Instala las bibliotecas declaradas.
Ejecutar pruebas de API y modelo	Ejecuta pytest sobre el backend y el modelo.
Construir imagen Docker	Construye la imagen mediante el Dockerfile de FaseII.
Probar contenedor construido	Arranca el contenedor y comprueba los endpoints HTTP, incluida la disponibilidad y una inferencia.

La tercera ejecución visible terminó en Success en 1 min 3 s. Las tres ejecuciones reportadas en el historial fueron exitosas. La primera y segunda ejecución generaron advertencias de compatibilidad de runtime; se actualizaron las acciones y se fijó ubuntu-24.04 en la tercera.

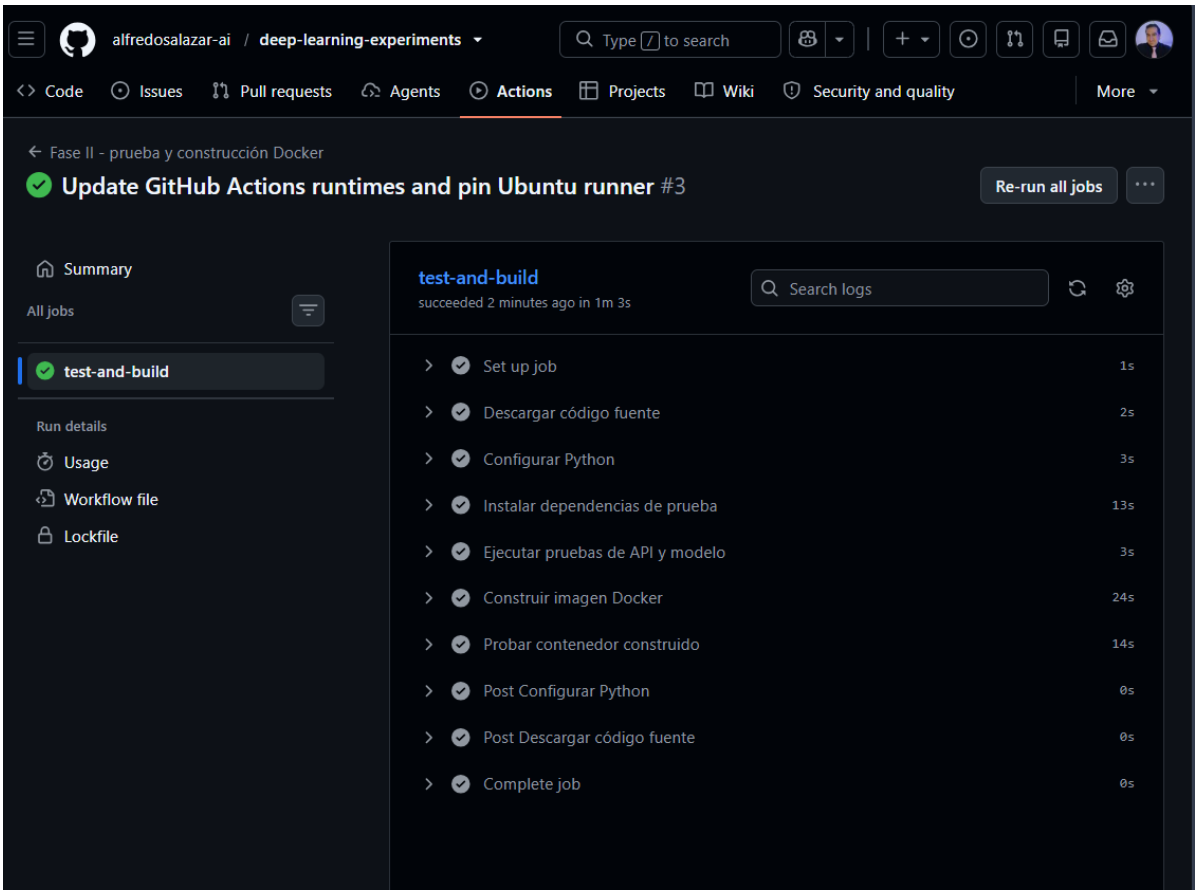


Figura 1. GitHub Actions: ejecución #3 exitosa después de actualizar los runtimes.

5. Configuración de Render

Campo	Configuración utilizada
Tipo	Web Service
Nombre	chatbot-cima-fase2

Campo	Configuración utilizada
Source	alfredosalazar-ai/deep-learning-experiments
Branch	main
Language / Runtime	Docker
Region	Oregon (US West)
Root Directory	FaseII
Docker build context	Carpeta FaseII (contexto del servicio)
Dockerfile	Dockerfile dentro de FaseII
Instance Type	Free · $0 USD/mes, 0.1 CPU, 512 MB RAM
Health Check Path	/health
Auto-Deploy	On Commit
Environment Variables	No se configuraron variables adicionales
Registry Credential	No credential
Docker Command / Pre-Deploy Command	Vacíos; se utilizan las instrucciones del Dockerfile

La pantalla final de Render mostró el plan Free seleccionado. No se eligió el plan de USD 7/mes. La configuración efectiva fue validada por el despliegue exitoso: Render encontró el Dockerfile, construyó la imagen y ejecutó el contenedor.

6. URLs de operación

Aplicación web: https://chatbot-cima-fase2.onrender.com/

Documentación interactiva Swagger: https://chatbot-cima-fase2.onrender.com/docs

Health Check: https://chatbot-cima-fase2.onrender.com/health

Repositorio GitHub: https://github.com/alfredosalazar-ai/deep-learning-experiments

7. Evidencia de despliegue exitoso

El log de despliegue correspondiente al commit 4933ce629b4afa90e9160a757648af51ddf87227 muestra:

- Clonado del repositorio y checkout de la rama main en el commit indicado.
- Construcción de la imagen basada en python:3.12-slim.
- Instalación de FastAPI 0.115.12, Uvicorn 0.34.0, scikit-learn 1.6.1, NumPy 1.26.4, SciPy 1.15.2 y joblib 1.6.0.
- Carga del modelo desde /app/modelos/Regresion_Logistica.joblib.
- Arranque de Uvicorn en 0.0.0.0:10000 y respuesta HTTP 200 de /health.
- Confirmación de Render: “Your service is live”.

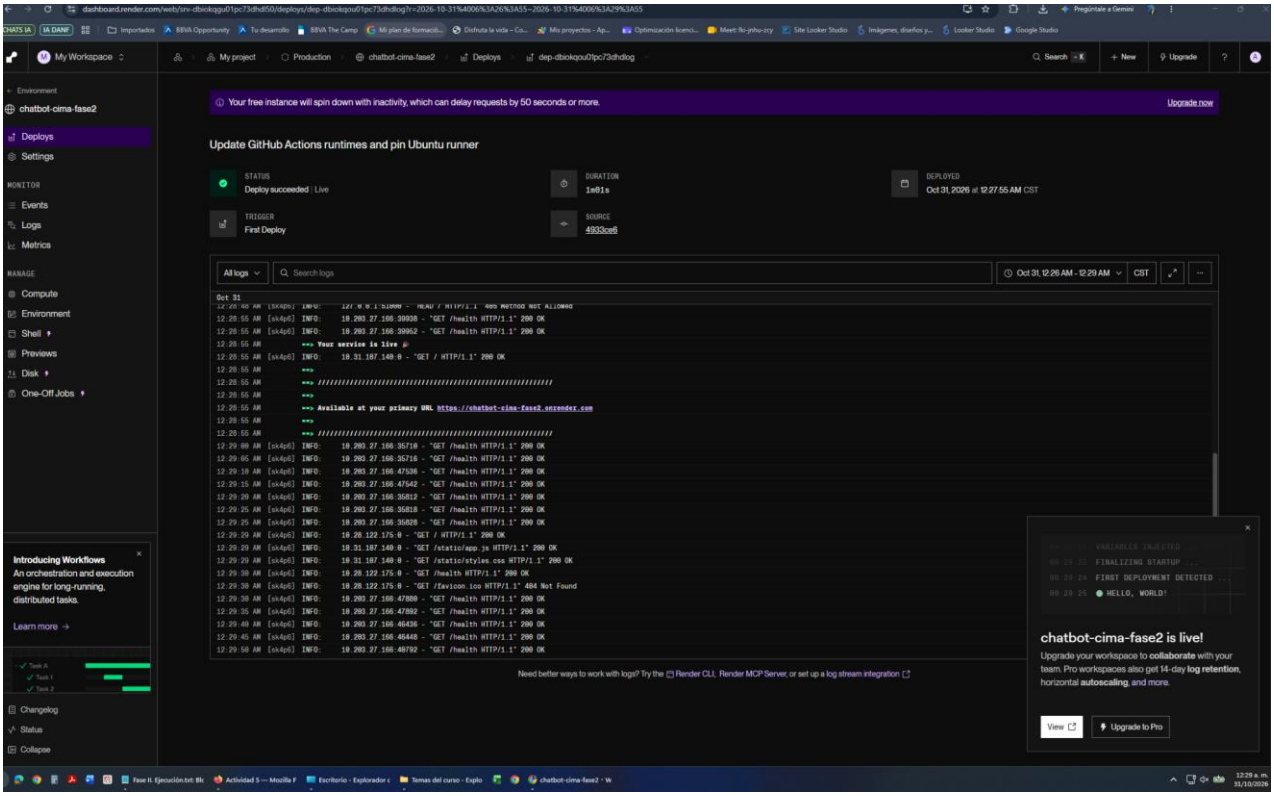


Figura 1. Extracto visual de los logs de Render.

8. Verificación posterior al despliegue

1. Abrir la URL pública de la aplicación y confirmar que carga el formulario.
2. Abrir /health y verificar HTTP 200, status=ok y model_loaded=true.
3. Abrir /docs y comprobar que estén disponibles GET /health y POST /predict.
4. Enviar una consulta sintética con el esquema {"texto": "..."} y revisar la respuesta JSON.
5. Confirmar que la decisión de enrutamiento sea coherente con el umbral y que las consultas no aceptadas automáticamente requieran revisión humana.
6. Conservar capturas de las solicitudes, códigos HTTP y respuestas.

9. Actualización del servicio

7. Realizar el cambio en la carpeta FaseII del repositorio GitHub.
8. Revisar el commit y confirmar que los archivos estén en la rama main.
9. Abrir GitHub → Actions y esperar el workflow “Fase II - prueba y construcción Docker”.
10. Verificar que las pruebas, el build Docker y la prueba del contenedor terminen en verde.
11. Render está configurado con Auto-Deploy On Commit; al detectar cambios relevantes en la carpeta del servicio, inicia una nueva construcción y despliegue.
12. Abrir Render → chatbot-cima-fase2 → Events/Logs y verificar “Deploy succeeded / Live”, carga del modelo y /health HTTP 200.

10. Suspensión por inactividad y reactivación

El servicio está en el plan Free. En este plan, Render suspende automáticamente la instancia después de un periodo de inactividad (aproximadamente 15 minutos sin tráfico entrante). La URL se conserva. Cuando se

realiza una nueva solicitud, Render reactiva la instancia; el primer acceso puede tardar más que una solicitud ordinaria.

- No es necesario mantener la aplicación consultando /health desde un monitor externo para la demostración académica.
- Para validar en una fecha posterior, abrir la URL pública y esperar el arranque si la instancia estaba suspendida.
- Después del arranque, repetir /health, /docs y al menos una solicitud POST /predict.
- No seleccionar un plan de pago. La configuración documentada es Free, USD 0/mes.

Disponibilidad El plan gratuito no ofrece disponibilidad continua ni persistencia de disco. No debe utilizarse para operaciones productivas, transacciones bancarias ni tratamiento de información sensible.

11. Seguridad y límites de uso

- Utilizar exclusivamente consultas sintéticas en pruebas y demostraciones.
- No introducir información bancaria real, datos personales, credenciales ni secretos.
- La predicción es informativa: no confirma transacciones ni autoriza pagos.
- La clasificación del modelo no reemplaza controles deterministas ni revisión humana.
- Mantener el plan gratuito y no habilitar recursos pagados sin autorización explícita.

12. Diagnóstico básico

Síntoma	Comprobación / acción
Render no inicia el contenedor	Revisar Build Logs; confirmar Root Directory FaseII y presencia del Dockerfile, requirements.txt, backend, frontend y modelos.
/health devuelve error	Revisar Runtime Logs; confirmar que el modelo se haya cargado y que el proceso Uvicorn esté activo.
Primera solicitud tarda	Puede ser el cold start del plan Free tras la suspensión por inactividad; esperar y reintentar.
/docs no carga	Confirmar que el servicio aparece Live y que la URL usa HTTPS y termina en /docs.
Favicon 404	Recurso visual accesorio no incluido; no impide usar la API.